# From netlist to verified model

Automatic real-number-model and testbench generation

for mixed-signal verification

Veylon Systems

**Abstract**

Real number models are how a mixed-signal block joins a digital regression suite — and writing one by hand is slow, keeping it faithful as the circuit changes is slower, and proving it is faithful is the part that usually never happens. **spice2rnm** closes the loop in one command: give it a SPICE netlist and it delivers a SystemVerilog real-number model, a UVM-MS verification environment, and the evidence that both agree with the circuit.

Three properties set it apart. Every golden value in every generated testbench is a *SPICE measurement* of your circuit, never the model's own equations. Every generated environment is proven able to fail, by injecting faults into the model and confirming the verdict flips. And the verification strategy adapts to what each block is *for* — a filter is checked as a frequency response, a duty-cycle corrector by where its edges land, a phase interpolator by phase per code.

This paper presents three case studies of increasing difficulty, with results produced end to end by the shipped tool in under six minutes on a laptop.



## Contents

# 1 The problem

A behavioural model has to be two contradictory things: fast enough to run in a digital regression, and faithful enough that passing it means something. Hand-written models drift from their circuits, and the drift is silent — the model still compiles, still simulates, still passes.

The usual defence is a hand-written testbench, which has the same problem one level up. Somebody decides what to check and how close is close enough, and those decisions are rarely written down next to the numbers they produced. When the circuit is revised, the model, the testbench, and the tolerances all have to be revisited by hand — so in practice they are not.

**spice2rnm** regenerates all three from the netlist, on demand. The model, the environment that checks it, and the evidence trail are outputs of one command, so they can never fall out of step with the circuit or with each other.

# 2 What you get from one command

```
spice2rnm dcc.cir --input-node clk --output-node vout \
          --emit-uvm-ms --spec dcc_spec.md
```

- **A SystemVerilog real-number model**, with the model structure chosen automatically to match the block's measured behaviour — including blocks whose dynamics move with signal level, where any fixed-response model is wrong across most of its range.

  Its ports are yours to choose (`--model-ports`). The computing core is emitted with plain `real` ports, because a `real` port is a *variable*: it depends on no package, no nettype declaration and no simulator extension, so it compiles standalone anywhere. Around that core the tool emits the boundary your flow uses — `wreal`, or a user-defined nettype taken from your own library (Section 9) — so the model presents the same ports as the models beside it.

  The core is untouched in every case — it is the file the equivalence check ran against — and a boundary costs nothing in fidelity: driven with identical stimulus, boundary and bare core agree exactly, worst difference 0 over 120 randomised samples across all three case-study models, and again on a library's own nettype. `wreal` is a simulator extension rather than IEEE 1800, so that boundary is a separate file the core does not depend on.

- **A UVM-MS verification environment** following the Accellera UVM-MS specification, built on IEEE 1800 — not on Verilog-AMS. Analog values travel on *user-defined nettypes* (IEEE 1800-2017 §6.6.7) with a generated resolution function, so multiple drivers on a net — an offset source, a noise injector, a second bridge — combine in the net type rather than in hand-edited testbench code; the default output contains no `wreal` and needs no AMS licence to run. Bridge and wrapper ports are `interconnect` (§6.6.8), inheriting the testbench's nettype instead of naming one, and scoreboard golden values are SPICE measurements of your circuit.

- **Equivalence evidence**: circuit and model driven with the identical stimulus, compared in the domain that matters for the block — waveform error for signal-path blocks, edge placement for clock-path blocks — with the aligned traces archived so any verdict can be re-derived from its own run directory.

An optional plain-language functional specification supplies what no netlist can: the block's intended operating range and its acceptance criteria (Section 8).

# 3 Case study 1 — Two-pole filter

A two-pole RC low-pass: the representative signal-path case. The generated model is checked four independent ways.

| | | |
|---|---|---|
| Chirp equivalence | 0.0290 against a 0.15 threshold | PASS |
| PRBS (independent stimulus) | 0.0097 normalised | PASS |
| UVM-MS, DC section | 51 checks, 0 failed | |
| UVM-MS, AC section | 19 checks, 0 failed | |
| worst magnitude | 0.0390 dB of a 0.25 dB tolerance | |
| worst phase | 2.70° of an 8.00° tolerance | |

The PRBS row matters more than it looks: the model is fitted against a chirp and then checked against a pseudo-random sequence it never saw. A model that had merely memorised its characterisation stimulus would be exposed here.

# 4 Case study 2 — Duty-cycle corrector

A comparator-based duty-cycle corrector whose dominant pole moves by a factor of 613,666 across its input range. Blocks like this defeat fixed-response modelling outright, and they defeat conventional AC-based verification too — which is exactly why they are usually the blocks whose hand-written models are trusted least.

## 4.1 A model that follows the circuit's dynamics

spice2rnm detects the level dependence during characterisation and builds a model whose dynamics are *scheduled against measured large-signal behaviour* — including the circuit's rise/fall timing asymmetry, which it measures at 95 ps and reproduces. At a 50% input duty the model matches the circuit's output duty to 0.000 percentage points.

| | | |
|---|---|---|
| Transient equivalence | 0.0769 against a 0.15 threshold | PASS |
| UVM-MS, DC section | 51 checks, 0 failed | |
| UVM-MS, duty section | 15 checks, 0 failed | |
| worst duty error | 0.314 pp of a 0.500 pp tolerance | |

## 4.2 Verified by function, not by formula

The generated environment for this block contains *no AC section* — by measurement, not by omission. A small-signal sweep linearises the circuit at one operating point, and a comparator in normal operation is never near that regime; checked that way, a demonstrably accurate model fails every point by more than 55 dB, telling you nothing about the model and everything about the mismatch between the check and the block.

Instead, the environment checks what a duty-cycle corrector is *for*. The tool drives the circuit with clocks at multiple rates and input duties, records the output duty from SPICE, and holds the model to those goldens — a table spanning 20 percentage points of measured behaviour, so a model that ignored its input could not reproduce it.

# 5 Case study 3 — Phase interpolator

A thermometer-coded phase interpolator has essentially no amplitude transfer at all — there is no frequency response worth fitting, and a voltage-domain comparison scores noise. Conventional flows stop here.

spice2rnm recognises the block's function and switches strategy: it measures phase per code, generates a delay-line model, and emits a *phase* verification environment — one measured golden per control code, no DC or AC section, because neither describes edge placement.

| | |
|---|---|
| Codes measured | 9 of 9 |
| Total traversal | −500.1 ps, monotonic |
| Mean step (1 LSB) | −62.51 ps |
| Worst DNL | −100.5 ps (−1.61 LSB) |
| UVM-MS phase section | 9 checks, 0 failed |
| worst | 3.26 ps of a 6.25 ps tolerance |

The tolerance is a tenth of an LSB, at the model's own quantisation floor — tight enough to fail on a real regression, never on rounding. The DNL figure is a property of the circuit, reported faithfully; the model's job is to reproduce it, and it does.

## 6 Results at a glance

| Case study | Environment | Checks | Failed |
|---|---|---|---|
| Filter | DC + AC | 51 + 19 | 0 |
| Duty corrector | DC + duty | 51 + 15 | 0 |
| Phase interpolator | phase per code | 9 | 0 |

All three case studies — characterisation, model generation, both simulators, and every generated environment — run end to end in 355 seconds on a laptop. spice2rnm's outputs are simulator-agnostic — plain IEEE 1800 SystemVerilog, tied to no vendor — and to keep every result in this paper reproducible by anyone, all of it was produced with universally available open-source simulators: **ngspice** on the analog side and **xezim** for the SystemVerilog and UVM-MS runs. No commercial licence was used anywhere in this document. The generated artifacts themselves — the models, the complete environments, and the evidence files, exactly as the tool wrote them — are published for inspection at

`https://github.com/vrajeshprakhya/spice2rnm-demos`

under `showcase/`, stamped with the tool version and run summary that produced them. What the repository holds:

```
spice2rnm-demos/
  demo_1_lpf.sh  demo_2_dcc.sh  demo_3_pi.sh  run_all.sh
                     the three case studies, runnable end to end
  harness/           8 measurement scripts the demos call
  showcase/          the generated artifacts, byte-for-byte
    lpf2/            the baseline case, 14 files:
      rc_lpf2_rnm.sv          the model, plain real ports
      rc_lpf2_rnm_wreal.sv    the same model, wreal ports
      rc_lpf2_ms_types_pkg.sv the generated nettype
      rc_lpf2_dms.sv          net <-> variable, interconnect ports
      rc_lpf2_bridge*.sv      MS bridge and its driving core
      rc_lpf2_proxy_pkg.sv    agent <-> bridge API
      rc_lpf2_ms_pkg.sv       agent, and the SPICE goldens
      rc_lpf2_ms_scoreboard.svh  the checks and tolerances
      rc_lpf2_ms_tb/test.svh  environment and test
      top_rc_lpf2_ms.sv       nets, DUT, bridge, test
      run_rc_lpf2_ms.sh       compiles and runs it
      result.json             fit, verdict, warnings
```

```
    dcc2/            same shape, 15 files: no AC section, adds
                     duty goldens in dcc_ms_pkg.sv and the run
                     log pipeline.txt
    pi/              same shape, 11 files: phase-per-code
                     goldens, no DC or AC section
    house_lib/       4 files, NOT generated: ng_ams_pkg.sv
                     declaring nettype real ng_anet, and two
                     hand-written models using it
    lpf2_house/      13 files: lpf2 again, joined to house_lib --
                     no types package, and rc_lpf2_rnm_ng_anet.sv
                     in place of the wreal boundary
    STAMP            date, tool version, and run summary that
                     produced the artifacts above
    refresh.sh       regenerates everything; the only way this
                     directory is ever updated
  whitepaper/        this document, source and PDF
```

And what it deliberately does not hold: the spice2rnm tool itself (the product is licensed separately; this repository is its evidence), and the build products of the runs — characterisation scratch, compiled simulator objects, DPI shims. `refresh.sh` filters those out, so the showcase is what a user would keep, not everything a run produces.

## 7 Why a green testbench is worth believing

Generated verification has a credibility problem: if the same tool writes the model and the test, what stops it from marking its own homework? Three design rules.

**Goldens are measurements, never the model.** Every golden value in every generated environment is a SPICE measurement of your circuit. Checking a model against the equations it implements would pass unconditionally; spice2rnm never does it.

**Every environment is proven able to fail.** The tool injects faults into the generated model and confirms the verdict flips — and the detection floor is *measured*, not asserted. For the duty-cycle corrector's environment:

| Injected fault | Detected by |
|---|---|
| comparator offset, 20 mV | DC and duty sections |
| dynamics slowed 10× | duty section (DC is structurally blind) |
| dynamics slowed 50× | duty section (DC is structurally blind) |

The middle column is the point: settled-DC checks — the whole of what many hand-written environments test — cannot see *any* error in a block's dynamics. The function-matched sections exist because of that blindness, and the fault sweep is re-run against the shipped model so the detection claims stay current.

**Sensitivity is pinned from both sides.** A check that fails everything is as useless as one that passes everything, so fault injection also confirms that errors *within* tolerance do not trip the environment.

## 8 Functional specifications as input

A netlist says what a circuit *is*. It never says what it is *for*, what rate it must run at, or how close is close enough. Hand those to the tool as a plain-language document — a datasheet excerpt, a requirements page — and it reads out the facts that shape verification:

| From the specification | Used for |
|---|---|
| operating clock range | where the environment drives the block |
| accepted input duty range | the stimulus sweep |
| duty accuracy requirement | the pass/fail tolerance |
| block function | which environment is generated |

On the duty-cycle corrector, the specification-driven run checks 35–65% input duty at the specified clock rates against the specification's own 1.0 pp requirement: 15 checks, 0 failed, worst 0.428 pp. PASS

**The specification is cross-checked, never trusted.** Every extracted fact is validated against what the circuit was actually measured to do, and the measurement wins. When the two disagree, that disagreement is surfaced as a finding:

```
SPEC CONFLICT: the spec says this block operates up to 4e+08 Hz, but
the fitted model is only valid below 1.195e+08 Hz -- 3.3x lower.
```

That is a coverage gap between a requirement and a characterisation, caught at model-generation time instead of at silicon correlation — and it is the kind of finding no amount of manual model review produces, because no one document contains both halves of it.

## 9 Joining an existing model library

A team with real-number models already has conventions — which user-defined nettype analog values travel on, which package declares it, what discipline their model ports use. Generated files that import their own freshly-invented nettype do not join that library; they sit beside it, each net scheme blind to the other, and the integration cost lands on the customer.

Point `--style-from` at the library and the output conforms: the environment's analog nets use *their* nettype, its files import *their* package, no types package of the tool's own is emitted, and the generated run script compiles their package file. A library whose models use `wreal` ports gets the wreal wrapper without being asked. Their timescale is reported but deliberately not adopted — generated delays are computed femtosecond values, and timescale is per compilation unit, so the directives interoperate unchanged.

**Reading a house style is a judgement, so it is read and then verified.** A library may declare several nettypes, only one of which is the living standard — and what says so is often a comment. Measured on a library that declares two, where the deprecated one happens to sort first alphabetically:

| | |
|---|---|
| mechanical scan | picks the deprecated nettype, and warns |
| model reading | picks the live one, quoting the deprecation notice |
| verification | re-checks the choice against the named file |

The model's stated grounds, verbatim from that run: *"line 15 marks `aaa_legacy_wire` deprecated ("kept only so ancient testbenches still compile. Do NOT use in new code"), … both models declare all analog ports as `acme_ewire`, confirming this is the live convention."* Nothing in that reading is taken on trust: the named nettype, package and file are checked against the file before use, and a claim the file does not support leaves the mechanical answer standing, out loud. Where the tool has a *measurement* — a count of which discipline the library's ports actually use — the count wins outright and a disagreement is recorded rather than adopted.

**The result is an environment that runs on the house net**, and it is published rather than described. The showcase carries the same filter twice: `lpf2/` generated normally, and `lpf2_house/` generated against `house_lib/` — a small hand-written library standing in for a customer's own. Diffing the two directories is the whole argument:

| | `lpf2/` | `lpf2_house/` |
|---|---|---|
| types package | emitted | **not emitted** |
| nets | `rc_lpf2_wire` | `ng_anet` |
| imports | the generated one | `ng_ams_pkg` |
| run script | compiles it | compiles the house package |
| model ports | plain `real` | `ng_anet`, and the environment instantiates it |
| the model | byte-identical | |
| the scoreboard | byte-identical | |

Conformance changes the boundary and nothing else — and the published environment runs from where it sits, against the library beside it: 51 DC and 19 AC checks, 0 failed, exactly the default environment's result.

Note what the environment instantiates in that second column. When the model is delivered behind a port boundary, the generated testbench drives *that* module rather than the core inside it, and the boundary's file joins the compile list — so what the evidence covers is what ships, not a component of it. Measured separately, a library resolving its net by averaging rather than summing conforms and passes the same way, and a library using `wreal` ports gets the wreal wrapper instead. Without `--style-from`, or against a directory with nothing recognisable in it, nothing changes — and the run says so.

## 10 AI where it helps, physics where it decides

spice2rnm uses language models at four points: proposing block boundaries in the device graph, classifying what a block is for, reading functional specifications, and reading an existing library's conventions (Section 9). All four follow one rule: **intent from the model, physics from the measurement**. Every proposal is validated against the measured circuit before it is used, and a proposal that fails validation is discarded — the run continues on measured defaults, and says so. Without an API key, every AI feature declines cleanly and the pipeline runs entirely on measurement.

## 11 Scope and requirements

- **Inputs**: SPICE netlists, flat or with subcircuits; optional plain-language functional specification; optionally an existing RNM library to take conventions from.
- **Outputs**: SystemVerilog real-number models, with the port discipline of your choice — plain `real` (portable anywhere), `wreal`, or your library's own nettype; UVM-MS 1.0 environments; equivalence reports with archived waveforms.
- **Toolchain**: a SPICE simulator for characterisation, and a digital simulator capable of running real number models — with UVM support for the generated environments. The generated models and environments are standard SystemVerilog and carry no vendor dependence, so either may be the one your flow already uses.
- **Block-level scope**: single-input single-output blocks and compositions of them, including current-mode blocks sharing a node. Hierarchical composition trades some accuracy for structure; the equivalence report states the composed figure separately from the per-block figures, so the trade is visible, not hidden.